

TopoGraph3D: An Executable Node-Graph System for Three-Dimensional Finite Element Analysis, Scientific Visualization, and Topology Optimization

Jinghao Chen^a

^aUniversity of Science and Technology of China, School of Mathematical Sciences; Student Number PB23010359, Hefei, 230026, Anhui, China

ARTICLE INFO

Keywords:

Scientific Visualization
Visual Programming
Hexahedral Finite Elements
Topology Optimization
Scalar Fields

ABSTRACT

TopoGraph3D is a Windows application for constructing and executing three-dimensional structural-analysis workflows as typed node graphs. A project combines structured domains of eight-node trilinear hexahedral (Hex8) elements, material and load definition, linear-static finite element analysis, solid isotropic material with penalization (SIMP) topology optimization, and interactive surface visualization in a single dataflow environment. The numerical core uses small-strain isotropic elasticity; the optimizer applies density filtering and an optimality-criteria update under a volume-fraction constraint. Solver fields are converted to display geometry by exterior-face extraction, triangulation, scalar assignment, model normalization, camera projection, centroid-depth ordering, and scalar coloring. The implementation is examined using the two bundled cantilever problems. The 384-element static case gives a compliance of 54.5284 N mm, a maximum displacement magnitude of 0.05621 mm, and a maximum element-centroid von Mises stress of 37.1558 MPa. The 216-element optimization case reaches the density-change stopping criterion after 21 evaluated designs at a physical volume fraction of 0.400, with final weighted compliance 317.906 N mm. Load scaling reproduces the expected linear-elastic relations, while a mesh comparison shows greater sensitivity in the local stress maximum than in the global responses. A threshold study quantifies the effect of the density threshold on the numbers of visible cells and boundary triangles.

1. Introduction

Structural optimization requires geometry, material data, boundary conditions, numerical solution, and visualization to share a consistent mesh and data model. TopoGraph3D represents these dependencies explicitly as a typed node graph. The graph is executable, and computed fields are mapped to boundary faces according to their nodal or elemental semantics.

Its main workflow begins with a structured three-dimensional domain, attaches material and load information, executes either linear-static finite element analysis (FEA) or solid isotropic material with penalization (SIMP) topology optimization, converts the result to a triangle surface, and displays the surface in an interactive viewer. The interface provides 20 node types, graph editing and stepping, diagnostics, and Scalable Vector Graphics (SVG) and Visualization Toolkit (VTK) export. Projects are stored as readable `.topopt` files in JavaScript Object Notation (JSON), and two cantilever projects are included as reference cases.

The implementation enforces four invariants:

1. a typed directed graph is validated and topologically ordered before execution;
2. finite-element and optimization results remain bound to the mesh from which they were computed;
3. volumetric fields are mapped to boundary triangles with explicit nodal-versus-elemental semantics;
4. displayed results and exported data are derived from the same graph state.

Figure 1 summarizes the resulting architecture.

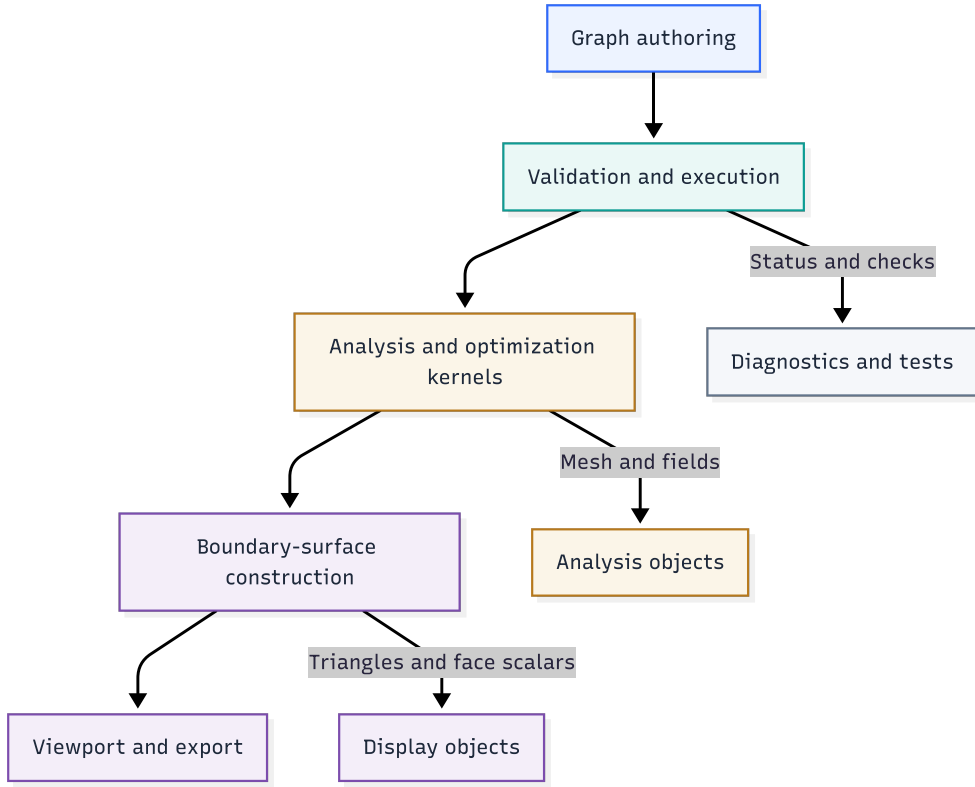


Figure 1: Software architecture and principal data objects of TopoGraph3D.

The numerical core and the display layer exchange explicit mesh and field objects. Model normalization and camera operations are confined to the display path, so they do not alter the coordinates used in finite element analysis or optimization.

2. Executable Node-Graph Model

2.1. Typed ports and graph validity

Let $G = (V, E)$ be the node graph. Each node stores a type, label, parameters, and ordered input and output ports. A directed connection

$$e = (u, i; v, j) \in E \quad (1)$$

sends output port i of node u to input port j of node v . Primitive port values are Number, Vector, and Boolean. Computational values include a general mesh (Mesh), a finite-element mesh (FEMesh), a material, a boundary condition, a load case, a finite-element result (FEResult), a density field, and the triangle-surface object consumed by the viewer.

A connection is accepted only when its endpoints exist, it runs from output to input, its data types agree or one endpoint is Generic, and the destination is unoccupied. The graph precheck also rejects cycles; each node checks its required inputs when evaluated. An execution order $\pi : V \rightarrow \{1, \dots, |V|\}$ therefore satisfies

$$(u, v) \in E \implies \pi(u) < \pi(v). \quad (2)$$

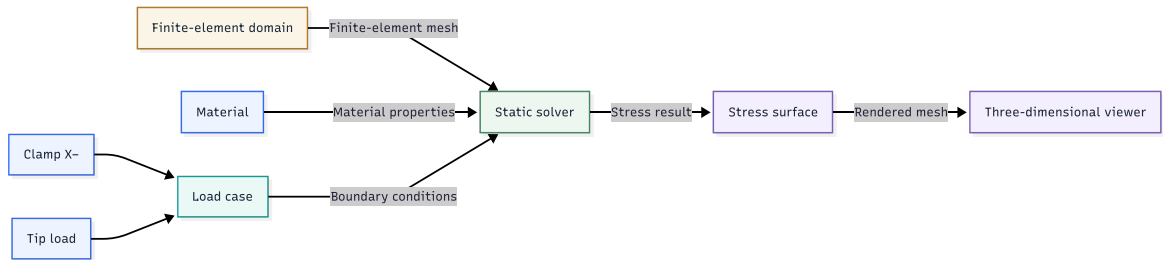
The implementation uses Kahn's topological-sorting algorithm. Nodes of zero indegree enter a queue, and removing a node decreases the indegrees of its successors. The algorithm returns all vertices exactly when the graph is acyclic; otherwise the residual subgraph contains a directed cycle.

2.2. Execution state and result binding

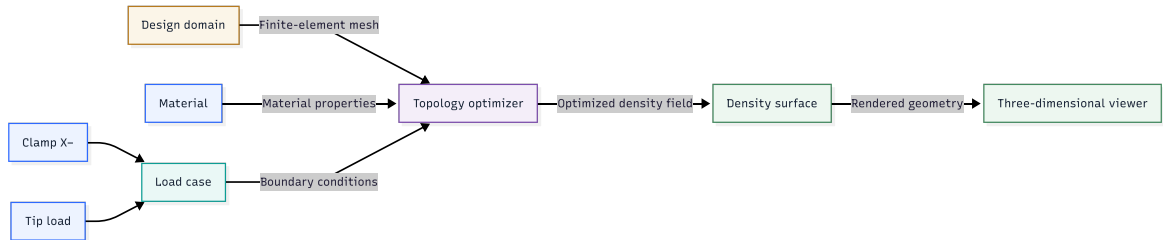
After validation, nodes run in topological order and cache their outputs by port. Run clears stale computed objects and executes the graph. Step evaluates one node rather than one solver iteration, so stepping onto a SIMP node completes the optimization. Reset clears results, the displayed three-dimensional field, and the step position but preserves graph structure and parameters.

A change in the upstream closure of an FEA or SIMP node clears the numerical-result cache; the implementation does not attempt dependency-level incremental recomputation. A 64-bit Fowler–Noll–Vo-style fingerprint of mesh coordinates and connectivity is stored with each finite-element or density result and checked before visualization, preventing accidental reuse after a mesh change.

Figure 2 shows the two bundled workflows. Both contain eight nodes and ten typed links. In each graph, the domain feeds the support and load selectors as well as the numerical node, while the load-case node combines the two boundary-condition objects. The workflows then diverge: the static solver sends its mesh and result to a stress-surface node, whereas the optimizer sends its density field to a density-surface node that also receives the design mesh.



(a) Linear-static workflow.



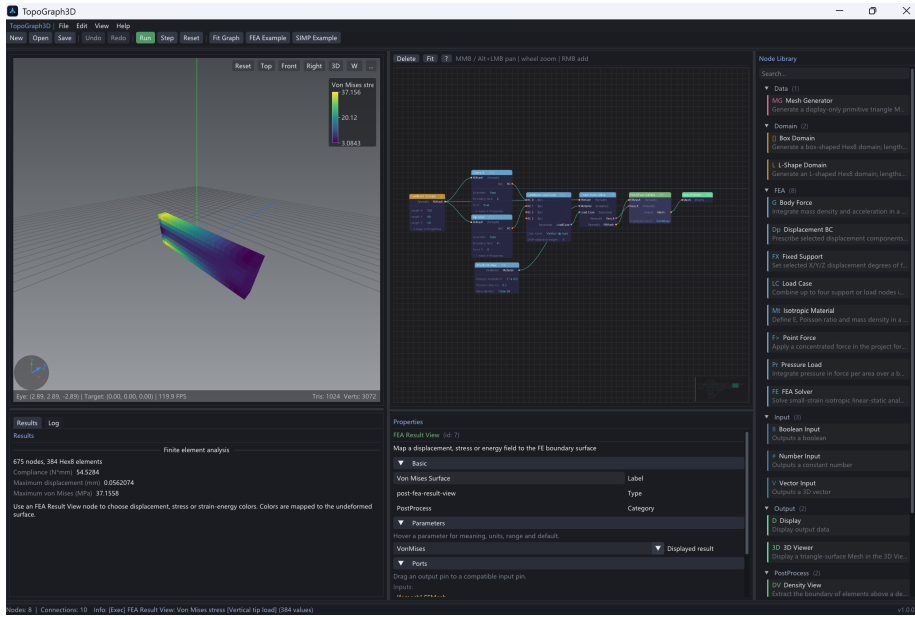
(b) Topology-optimization workflow.

Figure 2: Node graphs for the two bundled cantilever projects. Each workflow contains eight nodes and ten typed links.

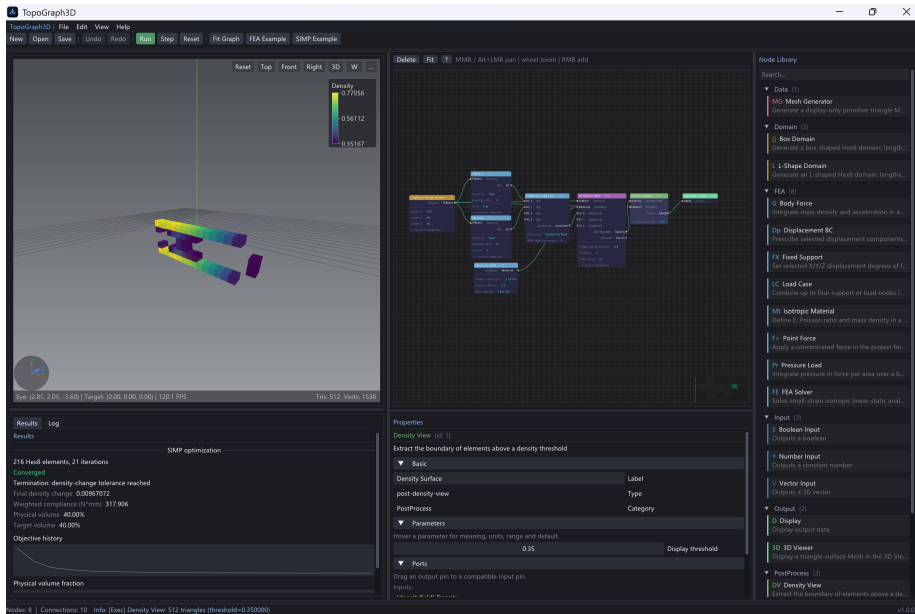
2.3. Interface organization and project persistence

The window contains a menu and toolbar, a three-dimensional viewport above the Results and Log tabs, a node canvas above the Properties panel, the searchable Node Library, and a status bar. Splitters allow the main panel proportions to be adjusted. The node canvas supports selection, link creation, panning, zooming, automatic framing, deletion, and a minimap. The viewer provides orbit, pan, zoom, standard views, wireframe control, ground grid and axis toggles, scalar colors, and SVG or VTK export.

Figure 3 shows the common interface layout for the two workflows. The graph and Results panels change with the analysis, while the viewer and editing controls retain the same organization.



(a) Bundled linear-static finite element analysis workflow.



(b) Converged topology-optimization workflow based on solid isotropic material with penalization (SIMP).

Figure 3: Unmodified Windows captures of the TopoGraph3D interface for the two bundled workflows.

The serializer records nodes and their canvas positions, connections, parameters, panel proportions, and orbit-camera state. Node-canvas pan and zoom are not serialized. Solver results are deliberately excluded: reopening a project reconstructs the workflow but requires a fresh execution. Unknown node types, invalid ports, duplicate input links, and cyclic graphs are rejected during loading rather than repaired silently.

The implementation uses C++17. Eigen supplies sparse linear algebra; the GLFW windowing library creates the Open Graphics Library (OpenGL) context; Dear ImGui and imnodes provide the interface and node editor; and nlohmann/json handles JSON project files. CMake defines the application and test targets.

3. Numerical Formulation

3.1. Assumptions and units

The current solver supports small-strain, homogeneous, isotropic, linear elasticity on conforming structured meshes of eight-node trilinear hexahedral (Hex8) elements. Box and L-shaped domains can be analyzed; imported surface models in the stereolithography (STL) and Wavefront OBJ formats are used only for visualization.

Quantities are used exactly as entered. The examples follow an N–mm–tonne–s convention, so Young’s modulus and stress are expressed in MPa = N/mm². The application does not provide dimensional conversion or a unit-consistency engine.

3.2. Trilinear eight-node hexahedral element

On the natural cube $(\xi, \eta, \zeta) \in [-1, 1]^3$, local node a has signs $(\xi_a, \eta_a, \zeta_a) \in \{-1, 1\}^3$ and shape function

$$N_a(\xi, \eta, \zeta) = \frac{1}{8}(1 + \xi_a\xi)(1 + \eta_a\eta)(1 + \zeta_a\zeta). \quad (3)$$

Geometry and displacement use the same interpolation:

$$\mathbf{x}(\xi, \eta, \zeta) = \sum_{a=1}^8 N_a \mathbf{x}_a, \quad \mathbf{u}(\xi, \eta, \zeta) = \sum_{a=1}^8 N_a \mathbf{u}_a. \quad (4)$$

With $J_{ij} = \partial x_i / \partial \xi_j$, the shape-function gradients satisfy $\nabla_{\mathbf{x}} N_a = \mathbf{J}^{-T} \nabla_{\xi} N_a$, from which the strain–displacement matrix is assembled. With engineering shear strain,

$$\boldsymbol{\epsilon} = [\epsilon_{xx}, \epsilon_{yy}, \epsilon_{zz}, \gamma_{xy}, \gamma_{yz}, \gamma_{xz}]^T = \mathbf{B} \mathbf{u}_e. \quad (5)$$

For Young’s modulus E and Poisson ratio ν , the Lamé parameters are

$$\lambda = \frac{E\nu}{(1+\nu)(1-2\nu)}, \quad \mu = \frac{E}{2(1+\nu)}. \quad (6)$$

The isotropic constitutive matrix is

$$\mathbf{D} = \begin{bmatrix} \lambda + 2\mu & \lambda & \lambda & 0 & 0 & 0 \\ \lambda & \lambda + 2\mu & \lambda & 0 & 0 & 0 \\ \lambda & \lambda & \lambda + 2\mu & 0 & 0 & 0 \\ 0 & 0 & 0 & \mu & 0 & 0 \\ 0 & 0 & 0 & 0 & \mu & 0 \\ 0 & 0 & 0 & 0 & 0 & \mu \end{bmatrix}. \quad (7)$$

The full-density element stiffness and volume are

$$\mathbf{K}_e^0 = \int_{\Omega_e} \mathbf{B}^T \mathbf{D} \mathbf{B} \, d\Omega, \quad V_e = \int_{\Omega_e} d\Omega. \quad (8)$$

Both integrals use a $2 \times 2 \times 2$ Gauss rule. An element is rejected if a Gauss-point Jacobian is non-positive or numerically degenerate. This formulation follows the standard isoparametric finite-element construction [1].

3.3. Loads, constraints, and response quantities

A point-force node can interpret its vector as either a total resultant over the selected node set or a per-node value. In Total mode, the resultant is divided equally among selected nodes. Uniform pressure is integrated on boundary quadrilaterals with a 2×2 rule, and uniform body force is integrated over element volume with the same eight-point rule used for stiffness.

After assembly and partition into free and prescribed degrees of freedom,

$$\mathbf{K}_{ff} \mathbf{u}_f = \mathbf{f}_f - \mathbf{K}_{fc} \mathbf{u}_c. \quad (9)$$

The reduced matrix is diagonally equilibrated and solved with sparse LDL^T factorization, where L is unit lower triangular and D is diagonal. Pivots and the final residual are checked; singular or severely ill-conditioned systems therefore produce a diagnostic.

For an element stiffness multiplier α_e , the element compliance contribution and strain energy are

$$c_e = \alpha_e \mathbf{u}_e^T \mathbf{K}_e^0 \mathbf{u}_e, \quad U_e = \frac{1}{2} c_e, \quad C = \sum_e c_e. \quad (10)$$

Before accumulation, a negative computed quadratic form is clamped to zero. Stress is evaluated once at the element center. The reported von Mises value is

$$\sigma_{\text{vm}} = \sqrt{\frac{1}{2} \left[(\sigma_{xx} - \sigma_{yy})^2 + (\sigma_{yy} - \sigma_{zz})^2 + (\sigma_{zz} - \sigma_{xx})^2 + 6(\tau_{xy}^2 + \tau_{yz}^2 + \tau_{xz}^2) \right]}. \quad (11)$$

No nodal extrapolation, stress recovery, or a posteriori error estimate is performed.

3.4. Topology optimization by solid isotropic material with penalization

Let $x_e \in [\rho_{\min}, 1]$ be the design variable and ρ_e the physical density after the selected filter. The implemented stiffness interpolation is

$$\mathbf{K}_e(\rho_e) = [\varepsilon + (1 - \varepsilon)\rho_e^p] \mathbf{K}_e^0, \quad \varepsilon = 10^{-9}, \quad (12)$$

where p is the penalty exponent. For active load cases $\ell = 1, \dots, L$, the optimization problem is

$$\begin{aligned} \underset{\mathbf{x}}{\text{minimize}} \quad & C(\mathbf{x}) = \sum_{\ell=1}^L w_\ell \mathbf{u}_\ell^T \mathbf{K}(\boldsymbol{\rho}) \mathbf{u}_\ell, \\ \text{subject to} \quad & \mathbf{K}(\boldsymbol{\rho}) \mathbf{u}_\ell = \mathbf{f}_\ell, \\ & \sum_e V_e \rho_e \leq f_V \sum_e V_e, \\ & \rho_{\min} \leq x_e \leq 1, \\ & x_e = \rho_e = 1 \quad (e \in \mathcal{P}_s), \\ & x_e = \rho_e = \rho_{\min} \quad (e \in \mathcal{P}_v). \end{aligned} \quad (13)$$

Here $w_\ell \geq 0$, at least one weight is positive, f_V is the target physical-volume fraction, and $\mathcal{P}_s$ and $\mathcal{P}_v$ denote optional passive-solid and passive-void sets. The implementation rejects any load case containing a body force and any positive-weight load case containing a nonzero prescribed displacement because its compliance sensitivity assumes design-independent external loads. The SIMP interpolation follows the material-distribution approach of Bendsøe and Sigmund [2, 3].

For element-center distance d_{ij} and radius $r_{\min}$, the filter weight is

$$H_{ij} = \max(0, r_{\min} - d_{ij}). \quad (14)$$

The density filter computes

$$\rho_i = \frac{\sum_j H_{ij} x_j}{\sum_j H_{ij}}, \quad (15)$$

Objective and volume derivatives are mapped to design variables through the transpose of this normalized operator. After filtering, passive-solid and passive-void densities are reset to 1 and $\rho_{\min}$, respectively. The alternative sensitivity filter leaves $\rho_i = x_i$ and smooths only the objective derivative; it is implemented but is not used in the reported cantilever case. The physical-density derivative before density-filter back-propagation is

$$\frac{\partial C}{\partial \rho_e} = -p(1 - \varepsilon)\rho_e^{p-1} \sum_{\ell=1}^L w_\ell \mathbf{u}_{e,\ell}^T \mathbf{K}_e^0 \mathbf{u}_{e,\ell}. \quad (16)$$

Filtering addresses mesh-dependent numerical patterns but does not convexify the problem or guarantee a global optimum [4, 5].

In the standard active-volume branch, free variables are updated with an optimality-criteria (OC) step,

$$x_e^{\text{new}} = \text{clip} \left(x_e \sqrt{-\frac{\partial C / \partial x_e}{\Lambda \partial V / \partial x_e}}, \max(\rho_{\min}, x_e - m), \min(1, x_e + m) \right), \quad (17)$$

where $m = 0.2$ and the multiplier Λ is found by bisection. If the move limits make the volume constraint inactive or unattainable in the current update, the maximum or minimum move-limit candidate is returned directly. Passive-solid and passive-void cells remain fixed, and their volume is included in the feasibility check. After at least six evaluations, the run stops when the maximum physical-density change between the current evaluated design and the next proposed update is below 0.01, or when the requested iteration limit is reached. History values are recorded at the final evaluated design. The update uses the density-based OC structure described in compact two- and three-dimensional SIMP implementations [6, 7].

4. Graphics and Scientific Visualization

4.1. Boundary extraction and triangulation

The analysis mesh is volumetric, while the viewer consumes triangles. For each Hex8 element, the six quadrilateral faces are represented by a canonical key obtained by sorting the four global node identifiers. Let $m(f)$ be the number of elements incident on canonical face f . For a conforming mesh,

$$f \text{ is exterior} \iff m(f) = 1. \quad (18)$$

Interior faces have multiplicity two and are discarded. Each retained quadrilateral (v_0, v_1, v_2, v_3) is split consistently into (v_0, v_1, v_2) and (v_0, v_2, v_3) . The triangle normal is computed from

$$\mathbf{n} = \frac{(\mathbf{v}_1 - \mathbf{v}_0) \times (\mathbf{v}_2 - \mathbf{v}_0)}{\|(\mathbf{v}_1 - \mathbf{v}_0) \times (\mathbf{v}_2 - \mathbf{v}_0)\|_2}. \quad (19)$$

Field semantics are preserved during this conversion. An element quantity, such as center von Mises stress or density, is copied to both triangles of every contributing exterior face. A nodal quantity uses the mean of the four face-node values,

$$s_f = \frac{1}{4} \sum_{a \in f} s_a. \quad (20)$$

Both triangles receive the same face-constant scalar value. Their displayed red–green–blue (RGB) values may still differ because lighting is evaluated from each triangle normal. No nodal stress recovery or continuous within-triangle interpolation is applied.

Figure 4 distinguishes the two export paths. VTK branches after boundary triangulation and scalar assignment and writes unprojected triangle geometry and cell data. The viewport and SVG paths additionally apply display normalization, camera projection, painter ordering, and shading.

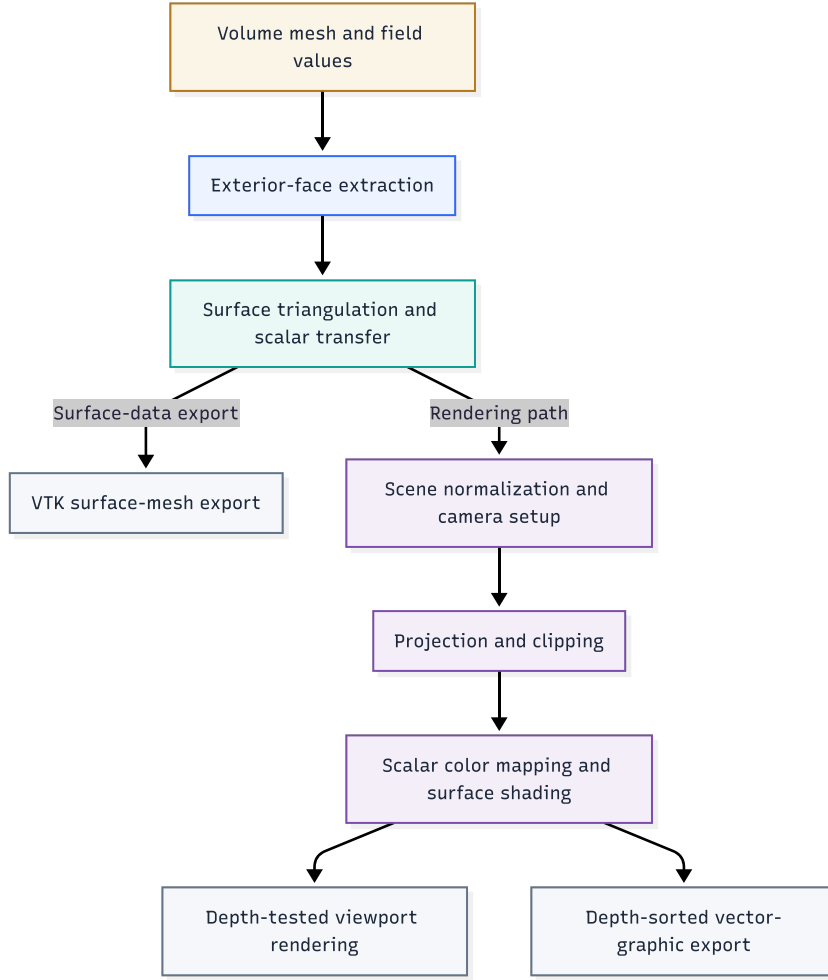


Figure 4: Conversion of volumetric fields to viewport, Scalable Vector Graphics (SVG), and Visualization Toolkit (VTK) outputs. The viewport is rasterized through a Dear ImGui draw list using centroid-depth ordering.

Because VTK branches before projection, it preserves model coordinates and scalar data for external processing, whereas SVG records the current camera-dependent view.

4.2. Normalization, orbit camera, and projection

Imported or generated geometry is translated to the center of its axis-aligned bounding box and scaled by the box diagonal. If c is the box center and $d = \sqrt{L_x^2 + L_y^2 + L_z^2}$, the normalized point is

$$\hat{\mathbf{x}} = \frac{5}{\max(d, 0.001)}(\mathbf{x} - \mathbf{c}). \quad (21)$$

The lower bound protects degenerate display geometry. This operation places models of different physical size in a comparable view volume without changing the analysis coordinates used by the numerical solver.

The implemented view transform applies yaw ψ followed by pitch θ :

$$\mathbf{x}_v = \mathbf{R}_x(\theta)\mathbf{R}_y(\psi)(\hat{\mathbf{x}} - \mathbf{t}), \quad (22)$$

where $\mathbf{t}$ is the camera target. With camera distance D , focal constant $f = 4.5$, viewport dimensions (W, H) , screen scale $q = 0.08 \min(W, H)$, and image center (c_x, c_y) , the perspective map is

$$x_s = c_x + qf \frac{x_v}{D + z_v}, \quad y_s = c_y - qf \frac{y_v}{D + z_v}. \quad (23)$$

The interface uses $(c_x, c_y) = (0.5W, 0.55H)$. A triangle is omitted when any vertex satisfies $D+z_v \leq 0.1$; no geometric clipping is performed at that plane.

4.3. Visibility, lighting, and scalar color

The viewer orders triangles by the view-space depth of their centroids and draws them from larger to smaller depth. This inexpensive rule is also used for SVG export, but it is not a depth buffer: intersecting triangles, cyclic overlap, and large triangles with misleading centroids can be ordered incorrectly.

A normalized scalar

$$t = \text{clip}\left(\frac{s - s_{\min}}{s_{\max} - s_{\min}}, 0, 1\right) \quad (24)$$

is mapped by piecewise-linear interpolation through five fixed RGB stops. A constant field is assigned $t = 0.5$. The control points progress from dark violet through blue and green to yellow. This RGB-interpolated map is not perceptually uniform.

Scalar-colored surfaces use a bounded lighting modulation. If $c(t)$ is the color-map output and b is a bounded combination of ambient, diffuse, fill, and specular terms, the displayed color is

$$c_{\text{display}} = (0.84 + 0.16b)c(t), \quad 0 \leq b \leq 1. \quad (25)$$

The 0.84–1.00 multiplier limits, but does not eliminate, shading-induced differences between faces carrying the same scalar value. Non-scalar meshes use the full lighting term.

SVG export records the current camera projection, surface style, face colors, and legend, but omits interface controls, the ground grid, and axes. VTK export writes the displayed triangle soup, face normals, and cell scalars as legacy text-based PolyData; vertices are repeated by triangle. Neither export is the original Hex8 volume mesh.

5. Verification and Results

5.1. Computational setup and reproducibility

Figures 1–4 summarize the implementation, interface, and bundled project files. Figures 5–10 use data stored in comma-separated values (CSV) files and exported by a C++ utility linked to the application’s finite element analysis solver (FEMSolver) and topology-optimization solver (TopOptSolver). All 16 numerical test groups and the independent CommandHistory test passed for the reported build.

5.2. Linear-static cantilever

The bundled FEA project models a $120 \times 40 \times 10$ mm beam with a $24 \times 8 \times 2$ mesh, giving 384 Hex8 elements and 675 nodes. The x -minus face is fixed in all three directions. The x -plus face carries the total force $(0, -1000, 0)$ N. Total means that 1000 N is the resultant over the selected face, not the load applied to each node. The material parameters are $E = 210000$ MPa and $\nu = 0.3$.

Table 1

Reference results for the bundled linear-static cantilever.

Quantity	Value	Unit
Compliance C	54.528412	N mm
Maximum displacement magnitude	0.0562074	mm
Maximum element-center von Mises stress	37.15578	MPa

The three quantities in Table 1 define the reference state used in the field, scaling, and mesh checks below. The displacement is small relative to the beam depth, which is consistent with the linearized kinematic assumption for this example.

Figure 5 shows the computed element-center stress. At each x – y cell, the plotted value is the larger result from the two elements through the thickness. The high values occur near the fixed end and the outer bending fibers. This is consistent with cantilever bending; the local maximum remains mesh dependent, as examined below.

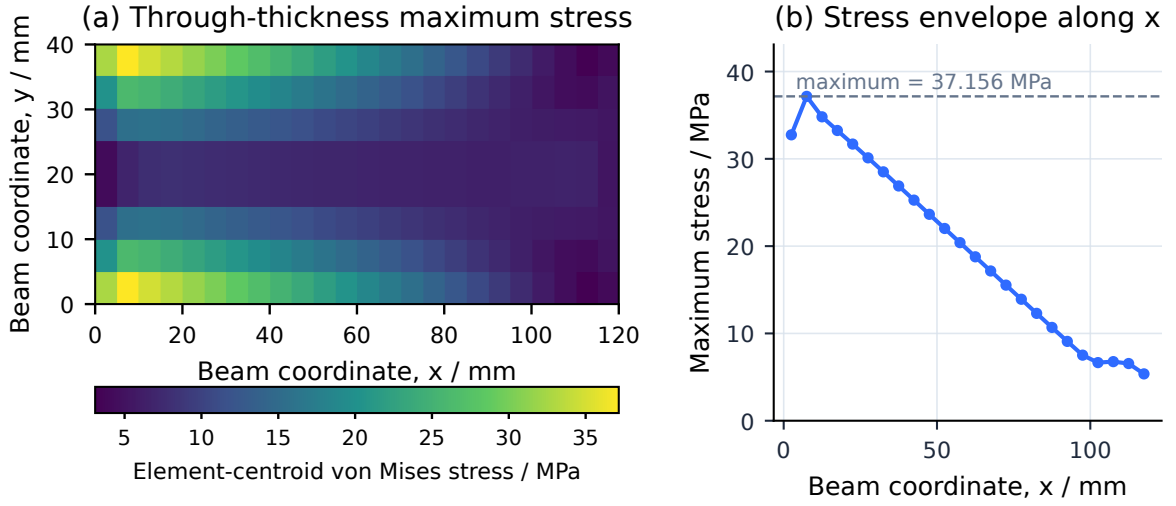


Figure 5: Element-center von Mises stress for the reference cantilever. Panel (a) gives the larger value across the two thickness layers at each x - y cell; panel (b) gives the cross-sectional maximum at each x position.

5.3. Load-scaling identity

With homogeneous prescribed displacements, a fixed linear system satisfies $\mathbf{u} \propto F$, $\boldsymbol{\sigma} \propto F$, and $C = \mathbf{f}^T \mathbf{u} \propto F^2$. The total load was evaluated at 250, 500, 750, and 1000 N without changing the mesh, material, or support.

Table 2

Load-scaling results for the cantilever model.

F / N	C / N mm	$u_{\max}$ / mm	$\sigma_{\max}$ / MPa
250	3.408026	0.0140518	9.28895
500	13.632103	0.0281037	18.57789
750	30.672232	0.0421555	27.86684
1000	54.528412	0.0562074	37.15578

The values in Table 2 follow the predicted powers of the load to the displayed precision. The normalized quantities in Figure 6 consequently collapse onto the same unit-slope line: halving the load halves displacement and stress and reduces compliance to one quarter.

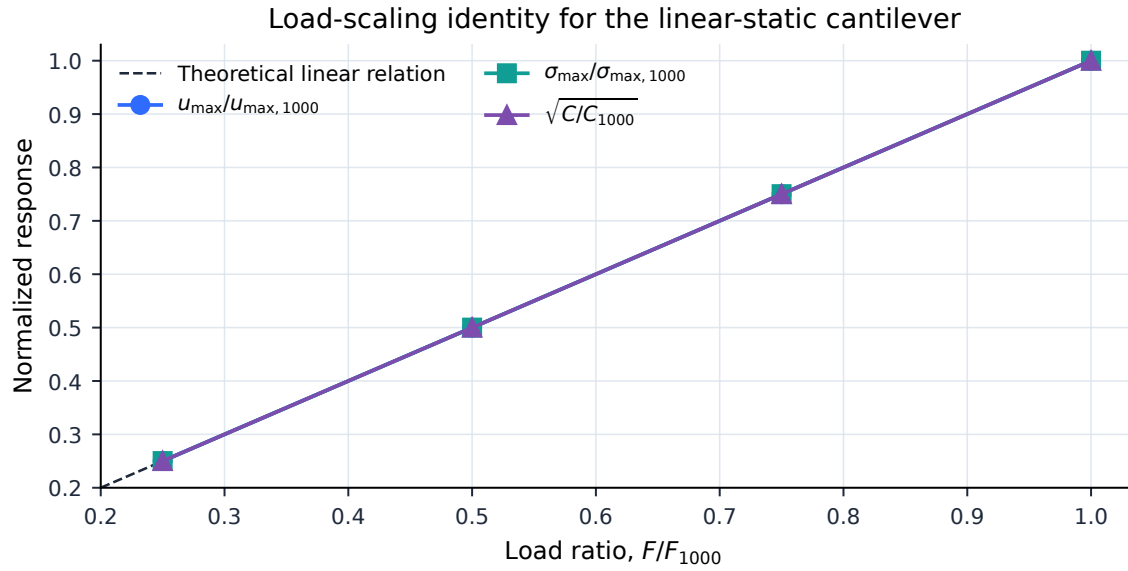


Figure 6: Linear-elastic scaling check. The square root of normalized compliance is plotted so that all three theoretical relations share the same unit-slope line.

5.4. Three-level mesh check

Three regular meshes were solved with identical physical dimensions and load. Table 3 shows that increasing the mesh from 384 to 3072 elements raises compliance and maximum displacement magnitude by approximately 1.01% and 1.06%, respectively, relative to the 384-element results. The maximum element-center stress rises from 37.16 to 41.49 MPa, so the local quantity remains more sensitive than the global responses.

Table 3

Regular-mesh refinement results.

Mesh	Elements	C / N mm	$u_{\max}$ / mm	$\sigma_{\max}$ / MPa
$12 \times 4 \times 1$	48	52.68564	0.0542239	29.3621
$24 \times 8 \times 2$	384	54.52841	0.0562074	37.1558
$48 \times 16 \times 4$	3072	55.07846	0.0568035	41.4893

Figure 7 makes the contrast visible: the global responses vary modestly between the two finer meshes, whereas the element-center stress remains more sensitive to refinement.

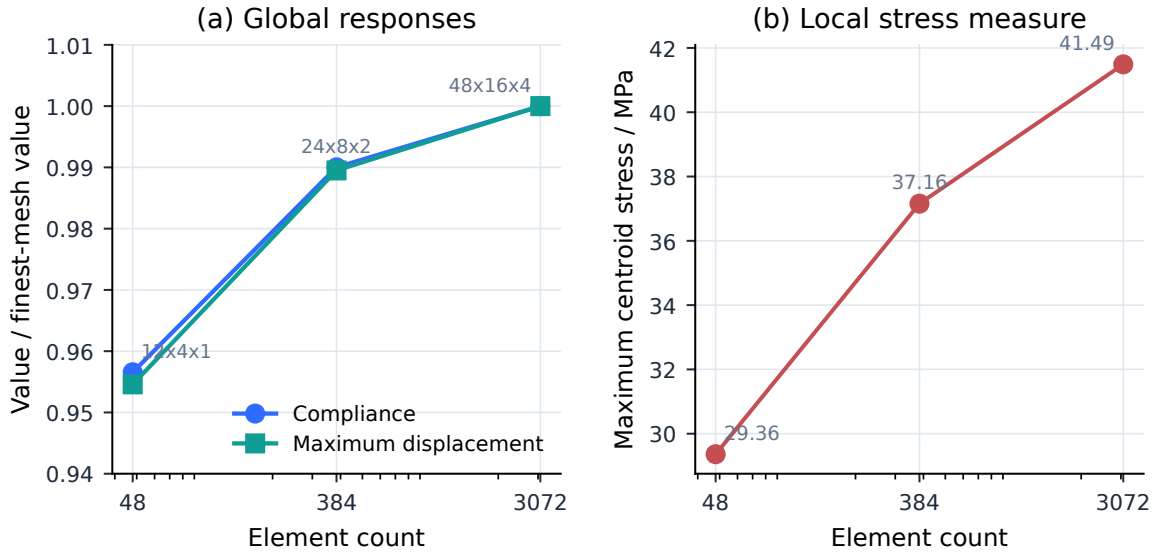


Figure 7: Three-level mesh comparison of compliance, displacement, and element-center stress. Global responses are normalized by the finest tested mesh.

5.5. Topology-optimized cantilever

The optimization project uses the same beam dimensions, material, support, and total-load semantics, with an $18 \times 6 \times 2$ design mesh containing 216 elements and 399 nodes. The parameters are $f_V = 0.40$, $p = 3$, $r_{\min} = 12$ mm, $\rho_{\min} = 0.001$, density filtering, and a maximum of 80 evaluated designs. No passive region is used.

Table 4

Solid isotropic material with penalization (SIMP) results for the bundled cantilever example.

Quantity	Value
Solved designs	21
Final weighted compliance	317.906 N mm
Physical volume fraction	0.400000006
Maximum proposed physical-density change	0.009671
Termination	Density-change criterion reached

Table 4 shows that the density-change criterion, rather than the iteration limit, terminated the run while the prescribed physical-volume fraction was maintained.

The through-thickness mean density field in Figure 8 concentrates material near the upper and lower beam regions and around the fixed end. At display threshold $\tau = 0.35$, 100 of 216 three-dimensional cells are retained. The thresholded geometry retains block faces; no surface smoothing is applied.

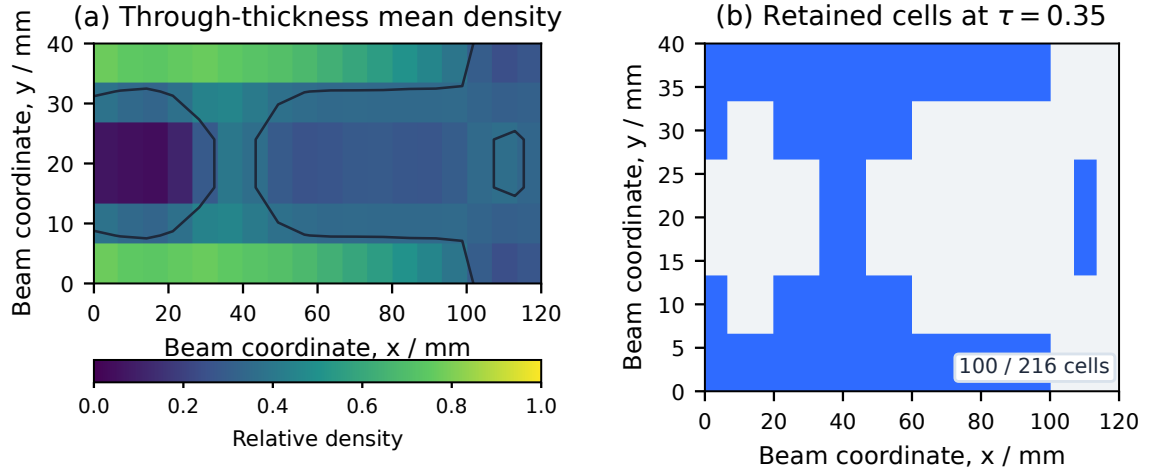


Figure 8: Final physical-density field. Panel (a) averages the two through-thickness layers and marks the $\rho = 0.35$ contour; panel (b) shows the corresponding retained-cell projection.

The first history entry is the initialized uniform-density design. Over 21 evaluated designs, the objective decreases from 844.342 to 317.906 N mm, a reduction of

$$\frac{844.342 - 317.906}{844.342} \times 100\% = 62.35\%. \quad (26)$$

Figure 9 shows that most of the objective reduction occurs during the first five evaluated designs, followed by progressively smaller improvements. The physical-volume fraction remains at its prescribed value of 0.40 to numerical precision throughout the run.

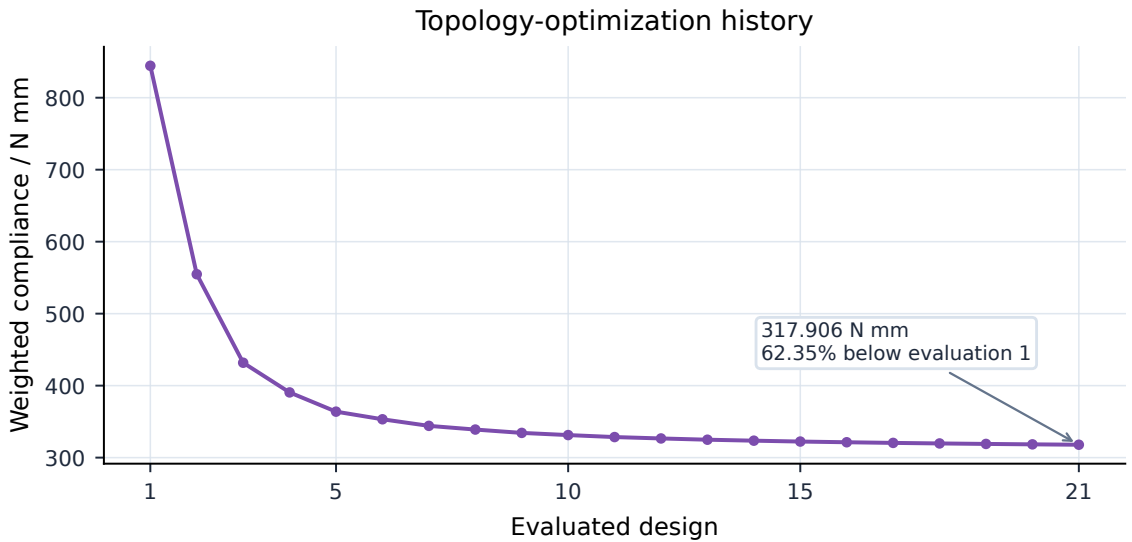


Figure 9: Weighted-compliance history for the 21 evaluated designs.

5.6. Display threshold and geometric complexity

The display threshold does not occur in the optimization problem in Equation (13). It acts only on the final field:

$$\mathcal{A}(\tau) = \{e : \rho_e \geq \tau\}, \quad T(\tau) = 2 |\partial \mathcal{A}(\tau)|, \quad (27)$$

where $\partial\mathcal{A}$ contains quadrilateral faces incident on exactly one retained cell. Although $|\mathcal{A}(\tau)|$ cannot increase with τ , the triangle count need not be monotone because removing an interior cell can expose new faces.

Figure 10 evaluates thresholds from 0.10 to 0.90 on the same density field. At $\tau = 0.35$, 100 cells yield 512 triangles; at $\tau = 0.50$, 52 cells yield 328 triangles. Changing the threshold alters visible geometry and the number of rendered primitives, but not the optimized density, objective, or volume fraction. For this Cartesian mesh, regular-grid neighbor counting gives the same boundary count as the canonical-face procedure used to define Equation (27).

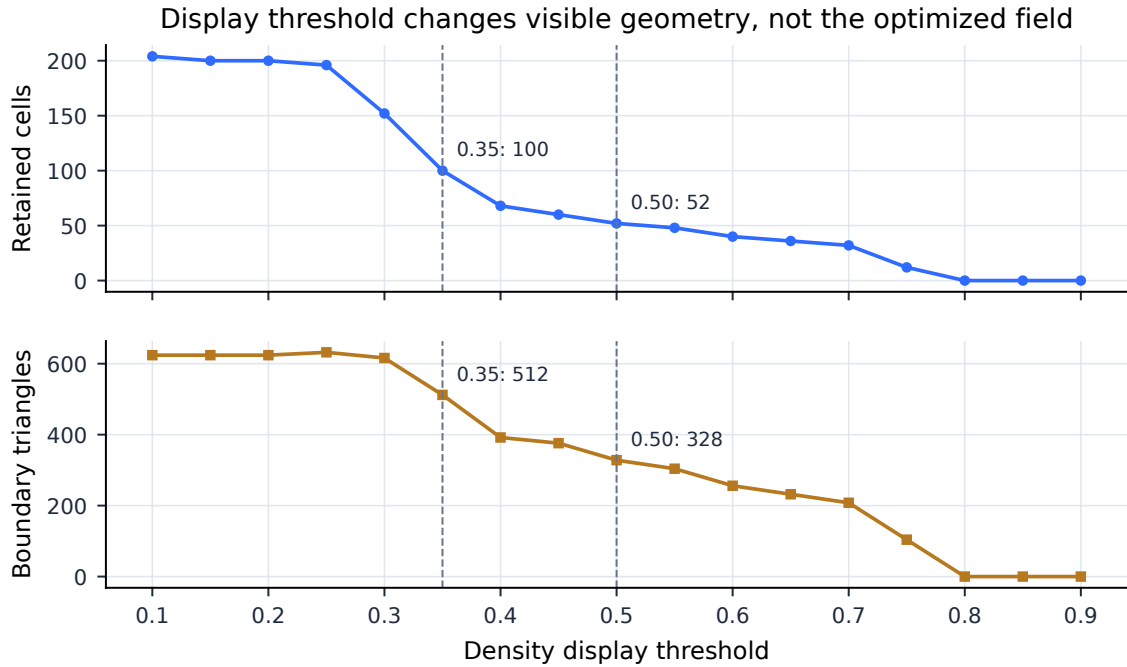


Figure 10: Effect of display threshold on retained cells and extracted boundary triangles for the final 216-cell density field.

5.7. Implementation tests

The automated tests cover Hex8 element operations, solver diagnostics, state invalidation, SIMP feasibility and termination, and the two reference parameter sets. The reference-case tests reconstruct the parameters in code; they neither parse the `.topopt` files nor execute the node graph. A separate core target exercises `CommandHistory` transitions. Table 5 summarizes the coverage of each test family.

Table 5

Summary of automated and numerical checks.

Check	Coverage	Result
Eight-node trilinear hexahedral (Hex8) elements	Regular and affine-transformed volume and stiffness, rigid modes, prescribed affine displacement	Passed
Finite element analysis solver	Scaling, isolated-node and unconstrained-model rejection, state invalidation, cantilever reference	Passed
Topology optimizer	Volume feasibility, passive regions, unsupported loads, termination, deterministic repeat	Passed
Report experiments	Four load levels, three regular meshes, and the display-threshold scan	Load identities and refinement trends verified
Command history	Execute, undo, redo, dirty, clean, and explicit <code>markDirty</code> transitions	Passed

Dedicated regression coverage is still needed for pressure and body loads, multi-load-case optimization, boundary-surface extraction, SVG and VTK export, and interaction with the graphical user interface.

6. Discussion

6.1. Visualization and workflow design

Together, the graphics stages determine how a volumetric solver field becomes a screen-space surface. Boundary extraction and scalar assignment determine the value represented by each face, whereas projection, depth ordering, and shading determine its appearance. Keeping these operations separate makes it possible to distinguish a change in numerical data from a change in view or display threshold.

The node graph provides the corresponding structure at the workflow level. Typed ports encode compatibility, directed links encode dependencies, and topological sorting produces the execution schedule. The intermediate mesh, field, and surface objects can therefore be inspected at the stages where their meaning changes.

6.2. Interpretation of the results

Load scaling agrees with the exact proportionalities of the linear system. The two finer meshes give similar compliance and displacement, whereas the element-center stress continues to increase, indicating that the local stress measure is not yet mesh independent. The threshold scan characterizes display geometry rather than solver accuracy. These conclusions are limited to the reported structured cantilever cases; the SIMP history also records convergence to the implemented stopping criterion rather than global optimality.

6.3. Limitations and future work

The principal limitations are as follows.

1. **Geometry and mechanics.** Analysis is limited to structured Box and L-shaped Hex8 domains under isotropic linear elasticity. Imported surfaces do not become analysis meshes, and there is no nonlinear or multiphysics solver.
2. **Field reconstruction.** Stress is sampled at element centers; nodal fields are reduced to one mean per exterior face; density geometry is a thresholded block boundary. There is no recovered nodal stress, deformed-shape overlay, cutting plane, error estimator, or smooth topology surface.
3. **Visibility.** The software-rendered viewer uses centroid-depth ordering and triangle rejection at the near plane. It has no depth buffer, geometric clipping, or back-face control, so difficult overlap cases can be rendered incorrectly.
4. **Execution and scale.** Solvers run synchronously, and filter neighborhoods are built by all-pairs center-distance checks. The interface limits generated domains to 20,000 elements, FEA to 10,000 elements, SIMP to 5,000 elements, and filter storage to 2,000,000 neighborhood entries.

A graphics processing unit (GPU) rendering path could replace centroid ordering with homogeneous clipping, back-face control, and a depth buffer. Vertex- or fragment-level field interpolation, deformed-shape overlays, recovered fields, cutting planes, and smooth density isosurfaces would expose variation omitted by the present face-constant surface. Cancellable background execution would keep the interface responsive during long SIMP runs, while spatial indexing would reduce the cost of constructing filter neighborhoods.

7. Conclusion

TopoGraph3D represents a structural-analysis workflow as an executable visual graph. Typed nodes and links connect structured domain generation, material and load definition, Hex8 finite elements, SIMP topology optimization, post-processing, and three-dimensional display. The implementation keeps computed fields associated with their source mesh and converts them to visible surfaces through explicit boundary extraction, triangulation, scalar assignment, camera projection, ordering, lighting, and export stages.

The static cantilever parameter set gives 54.5284 N mm compliance, 0.05621 mm maximum displacement magnitude, and 37.1558 MPa maximum element-center von Mises stress on 384 Hex8 elements. The 216-element optimization reaches its density-change criterion after 21 evaluated designs with final weighted compliance 317.906 N mm

at volume fraction 0.400. Linear load scaling, a three-level mesh check, threshold-to-triangle analysis, 16 numerical test groups, and a CommandHistory test provide reproducible checks within the tested configurations.

The reported results establish numerical and rendering behavior only for the tested configurations. Application to physical structures would require independent validation and broader solver, rendering, and interface testing.

Acknowledgements

I am grateful to Xiaoya Zhai for her guidance and support throughout this project.

References

- [1] Bathe, K. J. *Finite Element Procedures*, 2nd ed., K. J. Bathe, Watertown, MA, 2014.
- [2] Bendsøe, M. P. Optimal shape design as a material distribution problem. *Structural Optimization* 1 (1989) 193–202. <https://doi.org/10.1007/BF01650949>.
- [3] Bendsøe, M. P., Sigmund, O. Material interpolation schemes in topology optimization. *Archive of Applied Mechanics* 69 (1999) 635–654. <https://doi.org/10.1007/s004190050248>.
- [4] Sigmund, O., Petersson, J. Numerical instabilities in topology optimization: a survey on procedures dealing with checkerboards, mesh-dependencies and local minima. *Structural Optimization* 16 (1998) 68–75. <https://doi.org/10.1007/BF01214002>.
- [5] Bourdin, B. Filters in topology optimization. *International Journal for Numerical Methods in Engineering* 50 (2001) 2143–2158. <https://doi.org/10.1002/nme.116>.
- [6] Sigmund, O. A 99 line topology optimization code written in Matlab. *Structural and Multidisciplinary Optimization* 21 (2001) 120–127. <https://doi.org/10.1007/s001580050176>.
- [7] Liu, K., Tovar, A. An efficient 3D topology optimization code written in Matlab. *Structural and Multidisciplinary Optimization* 50 (2014) 1175–1196. <https://doi.org/10.1007/s00158-014-1107-x>.